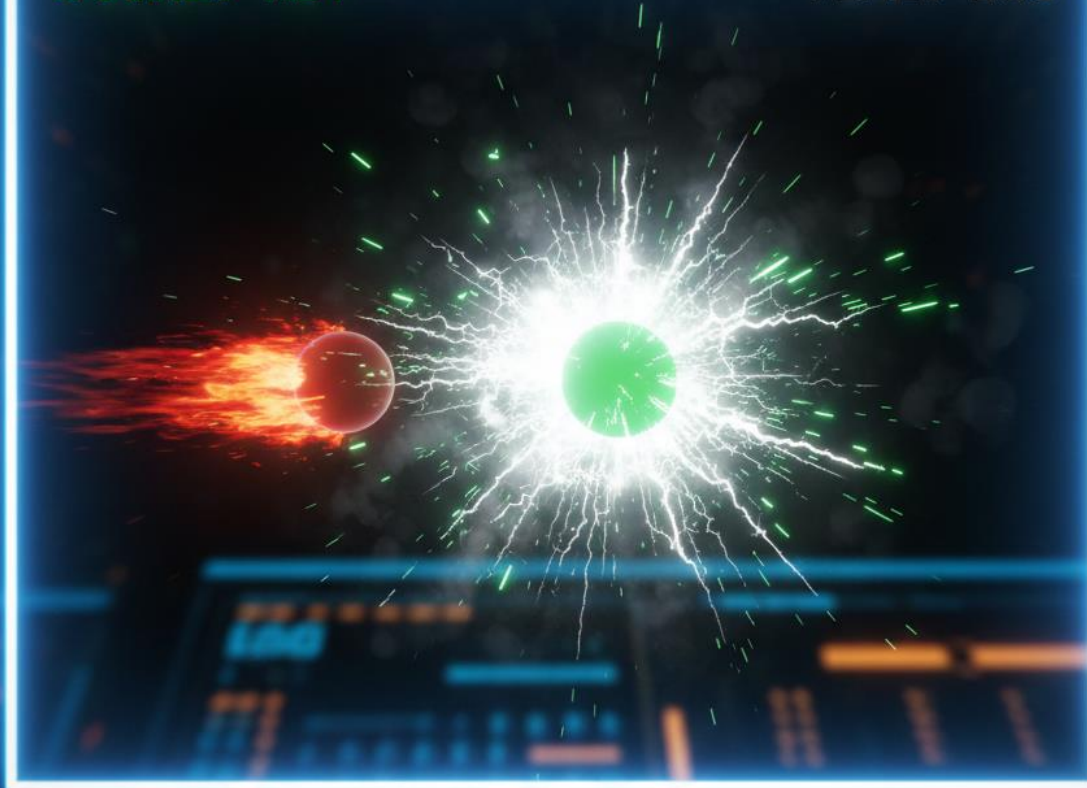


點點反應力遊戲

SCORE: HIT

TIME: 25s



組別：第二組

組員：41423113、41423144、



技術架構

- **Python 在瀏覽器執行：**

無需 JavaScript 基礎，直接用 Python 控制網頁。

- **非同步事件處理：**

使用 `browser.timer` 模組，讓多個目標球體各自擁有獨立的移動頻率。

- **DOM 即時渲染：**

動態生成 HTML 元素，並透過 Python 屬性（如 `t.style`）直接調整 CSS。

核心邏輯流程

- 自動化位移：

使用 `timer.set_interval` 為每個目標設定獨立的計時器，讓它們各自的速度在畫面內隨機跳動。

- 事件綁定：

透過 `t.bind("click", hit_target)` 監聽玩家動作。當玩家點擊目標時，會立即觸發分數計算與位置重置。

- 倒數計時系統：

利用遞迴呼叫 `check_time` 函數，每秒檢查剩餘時間。當 30 秒結束，程式會強制清除所有的 `interval`（計時器），並彈出視窗顯示最終戰績。

視覺回饋

- 球體動畫：

擊中後會觸發 `scale(0)` 的縮小動畫，模擬球體被「點破」的視覺效果。

- 畫面震動：

擊中目標時，整個 `game_area` 會產生隨機位移，增強物理撞擊感。

- 發光效果：

利用 CSS 的 `box-shadow` 為球體增加霓虹發光，提高在深色背景下的可辨識度。

遊戲流程

- 初始化：

遊戲啟動後，根據配置檔動態生成目標球。

- 主循環：

每個目標球擁有獨立的 interval，時間一到便隨機移動位置。

- 倒數計時：

系統每秒檢查剩餘時間，直到 GAME_DURATION 歸零。

- 結算：

遊戲結束時凍結所有動作，並彈出視窗顯示玩家的最終得分。

程式碼

```
from browser import document, timer, window
import random
import time

# --- 初始化變數 ---
GAME_DURATION = 30
score = 0
game_active = False
interval_handles = []
targets = []
game_over_time = 0

target_configs = [
    {"size": 60, "speed": 2000, "points": 1, "color": "#2ecc71", "z": "10"},
    {"size": 35, "speed": 1000, "points": 5, "color": "#e74c3c", "z": "20"}
]

# --- 建立 UI 介面 ---
game_area = document["brython_div1"]
game_area.style.position = "relative"
game_area.style.backgroundColor = "#1a1a2e"
game_area.style.overflow = "hidden"
game_area.style.border = "4px solid #333"

# 分數顯示
score_display = document.createElement("div")
score_display.style.cssText = """
    color: #00ff00; padding: 10px 20px; font-size: 40px;
    font-family: 'Courier New', Courier, monospace; font-weight: bold;
    text-shadow: 2px 2px #000; position: absolute; top: 0; left: 0; z-index: 100;
"""
score_display.text = "SCORE: 0"
game_area <= score_display

# 時間顯示
timer_display = document.createElement("div")
timer_display.style.cssText = """
    color: #ffffff; padding: 10px 20px; font-size: 24px;
    position: absolute; top: 0; right: 0; z-index: 100;
"""
game_area <= timer_display

# --- 核心邏輯 ---

def move_target(t):
    if not game_active: return
    size = int(t.getAttribute("data-size"))
    max_x = max(0, game_area.clientWidth - size)
    max_y = max(80, game_area.clientHeight - size)

    # 恢復狀態並移動位置
    t.style.opacity = "1"
    t.style.transform = "scale(1)"
    t.style.left = f"{random.randint(0, max_x)}px"
    t.style.top = f"{random.randint(80, max_y)}px"

def hit_target(event):
    global score
    if not game_active: return

    event.stopPropagation()
    event.preventDefault()

    target = event.target
    pts = int(target.getAttribute("data-points"))
    score += pts
    score_display.text = f"SCORE: {score}"

    # --- 視覺回饋效果 ---
    # 1. 球體縮小並淡出
    target.style.transform = "scale(0.2)"
    target.style.opacity = "0"
    # 2. 擴大光暈陰影
    target.style.boxShadow = f"0 0 40px 20px {target.style.backgroundColor}"

    # 延後 150ms 等動畫跑完再重新定位球體
    timer.set_timeout(lambda: move_target(target), 150)
```

```
# 時間顯示
timer_display = document.createElement("div")
timer_display.style.cssText = """
    color: #ffffff; padding: 10px 20px; font-size: 24px;
    position: absolute; top: 0; right: 0; z-index: 100;
"""
game_area <= timer_display

# --- 核心邏輯 ---

def move_target(t):
    if not game_active: return
    size = int(t.getAttribute("data-size"))
    max_x = max(0, game_area.clientWidth - size)
    max_y = max(80, game_area.clientHeight - size)

    # 恢復狀態並移動位置
    t.style.opacity = "1"
    t.style.transform = "scale(1)"
    t.style.left = f"{random.randint(0, max_x)}px"
    t.style.top = f"{random.randint(80, max_y)}px"

def hit_target(event):
    global score
    if not game_active: return

    event.stopPropagation()
    event.preventDefault()

    target = event.target
    pts = int(target.getAttribute("data-points"))
    score += pts
    score_display.text = f"SCORE: {score}"

    # --- 視覺回饋效果 ---
    # 1. 球體縮小並淡出
    target.style.transform = "scale(0.2)"
    target.style.opacity = "0"
    # 2. 擴大光暈陰影
    target.style.boxShadow = f"0 0 40px 20px {target.style.backgroundColor}"

    # 延後 150ms 等動畫跑完再重新定位球體
    timer.set_timeout(lambda: move_target(target), 150)
```

```
def check_time():
    global game_active, game_over_time
    rem = int(game_over_time - time.time())
    if rem <= 0:
        game_active = False
        timer_display.text = "TIME UP!"
        for h in interval_handles: timer.clear_interval(h)
        window.alert(f"【遊戲結束】\n最終得分: {score}")
    else:
        timer_display.text = f"TIME: {rem}s"
        timer.set_timeout(check_time, 1000)

def start_game():
    global game_active, game_over_time, score, interval_handles, targets
    for t in targets:
        try: t.remove()
        except: pass
    targets = []
    for h in interval_handles: timer.clear_interval(h)
    interval_handles = []

    score = 0
    score_display.text = "SCORE: 0"
    game_active = True
    game_over_time = time.time() + GAME_DURATION

    for cfg in target_configs:
        t = document.createElement("div")
        # 加入 transform 與 opacity 的過渡效果
        t.style.cssText = f"""
            width: {cfg['size']}px; height: {cfg['size']}px;
            background-color: {cfg['color']};
            position: absolute; border-radius: 50%;
            cursor: pointer; z-index: {cfg['z']};
            box-shadow: 0 0 15px {cfg['color']};
            transition: left 0.2s, top 0.2s, transform 0.15s, opacity 0.15s, box-shadow 0.1s;
            """
        t.setAttribute("data-points", cfg['points'])
        t.setAttribute("data-size", cfg['size'])
        t.bind("mousedown", hit_target)

        game_area <= t
        targets.append(t)
        move_target(t)

    h = timer.set_interval(lambda obj=t: move_target(obj), cfg['speed'])
    interval_handles.append(h)

    check_time()

timer.set_timeout(start_game, 200)
```

遊戲機制設計

屬性	一般目標 (綠色)	進階目標 (紅色)
大小	60px (較易擊中)	35px (較難擊中)
速度	2秒移動一次	1秒移動一次
分數	1 分(每點擊一次)	5 分

